Vulnerability Disclosure Report

Target Domain: aknu.edu.in / api.aknu.edu.in

Date of Discovery: July 2026

Reporter: [KARTHIK DORAPALLY / Security Researcher]

Executive Summary

During a routine security assessment of the web infrastructure, multiple vulnerabilities were identified within the API subdomain and main application architecture. These issues range from information disclosure via unauthenticated endpoints to high-severity injection and directory traversal risks. If exploited, these flaws could allow unauthorized access to sensitive student records, financial information, and administrative data.

Detailed Vulnerability Breakdown

1. Critical: Unauthenticated API Access

- **Affected Endpoint:** https://api.aknu.edu.in
- **Description:** The core API endpoint lacks authentication mechanisms. Unauthenticated requests can successfully enumerate data pools including gallery images, notifications, examination schedules, internal circulars, and timetables.
- **Impact:** High-volume information disclosure affecting institutional records.

2. Critical: Potential NoSQL Injection Risk

- **Affected Endpoint:** /results search form
- **Tech Stack:** Node.js + Express + MongoDB
- **Description:** Input handling on the student registration form does not appear to properly sanitize or bind input objects. If parameters are passed directly into database queries, a malicious actor could pass standard MongoDB logical operators to bypass validation rules.
- **Example Vector Syntax:**

JSON

```
{
 "registrationNumber": {"$gt": ""},
 "password": {"$gt": ""}
}
```

- **Impact:** Potential bypass of query logic, leading to bulk dumping of student records.

3. High: **Insecure Direct Object Reference (IDOR)**
   - **Affected Endpoints:** Gallery/Notification hash lookups and result query strings.
   - **Description:** The system references objects using their direct database keys or predictable parameters (such as sequential hall ticket/registration numbers). Without contextual session checks checking *who* is requesting the data, anyone can iterate through identifiers to view private data.
   - **Impact:** Mass enumeration of academic and personal student records.
4. High: **Path Traversal / Arbitrary File Disclosure**
   - **Affected Endpoint:** /api/notifications/file/
   - **Description:** The API handles file requests using path strings. If input paths are not strictly sanitized against relative navigation sequences (e.g., ../), an attacker may attempt to escape the designated web directory.
   - **Example Vector Syntax:**

Plaintext

```
GET /api/notifications/file/../../../[system_file_path]
```

   - **Impact:** Potential exposure of underlying system files or restricted configuration documents.
5. [redacted]: **Client-Side CAPTCHA Bypass**
   - **Affected Endpoint:** /results page
   - **Description:** The implementation utilizes a React single-page application (SPA) architecture where validation logic or tokens appear to remain static, reusable, or heavily exposed on the client side. Direct API calls can bypass the user interface check.
   - **Impact:** Enables automated brute-forcing or mass scraping of student databases using scripted queries.
6. [redacted]: **Permissive CORS Configuration**

- **Affected Endpoint:** Core API responses
- **Description:** Cross-Origin Resource Sharing (CORS) policies are overly broad, potentially allowing arbitrary external origins to interface with the API environment via cross-domain scripting.
- **Impact:** Increases vulnerability to client-side data leaks when users interact with external sites simultaneously.

7. **: Sensitive Document Exposure (Legacy Endpoints)**

- **Affected Paths:** /Exams/links/Notifications/ and /Exams/links/
- **Description:** While directory listing is suppressed (returning a blank 200 OK index), legacy or unlinked files remain active on the server. The following sensitive datasets were found exposed:
    - I B.Ed Student Data Format.xlsx (Contains student templates including national identity numbers, hall tickets, and active contact information).
    - Legacy financial documentation displaying institutional bank account information (Account No- 062611011000715).
- **Impact:** Direct leakage of Personally Identifiable Information (PII) and institutional financial details.

Recommended Remediation Steps

1. **Implement Robust Authentication:** Enforce session-based validation or JSON Web Tokens (JWT) across all sensitive paths hosted on api.example.edu.
2. **Sanitize NoSQL Queries:** Use Object Data Modelers (ODMs) like Mongoose, ensure inputs are strictly cast to strings, or use query sanitization middleware to strip out operator prefix characters like $.
3. **Enforce Access Controls:** Validate that the requesting session owns or has permission to view the specific object ID or hall ticket number being queried.
4. **Path Sanitization:** Use built-in path resolution libraries (like Node's path.resolve) to verify that requested files reside strictly within the intended public directory.
5. **Server-Side Validation:** Move CAPTCHA verification completely to the server side and invalidate tokens immediately after a single validation attempt.
6. **Restrict CORS Policies:** Explicitly define white-listed domain origins instead of returning wildcards or echoing arbitrary origin headers.
7. **Clean Legacy Assets:** Permanently remove or restrict public file-system access to obsolete Excel templates and financial logs.

NOTE - "Please let me know if you have a formal vulnerability rewards program or any guidelines regarding the recognition of security researchers who report findings responsibly."